

บทที่ 13

เทคนิคในการทำงานฝั่งไคลเอนต์

ในส่วนฝั่งไคลเอนต์นั้น จะรับผิดชอบในการรับข้อมูลต่างๆจากเซิร์ฟเวอร์ มาแสดงผลกราฟิกให้สวยงาม และทำการรับคำสั่งจากผู้เล่น ส่งให้เซิร์ฟเวอร์ประมวลผลต่อไป ฉะนั้นงานที่ไคลเอนต์รับบทหนักจริงๆ นั้น คือการแสดงผลกราฟิกนั่นเอง

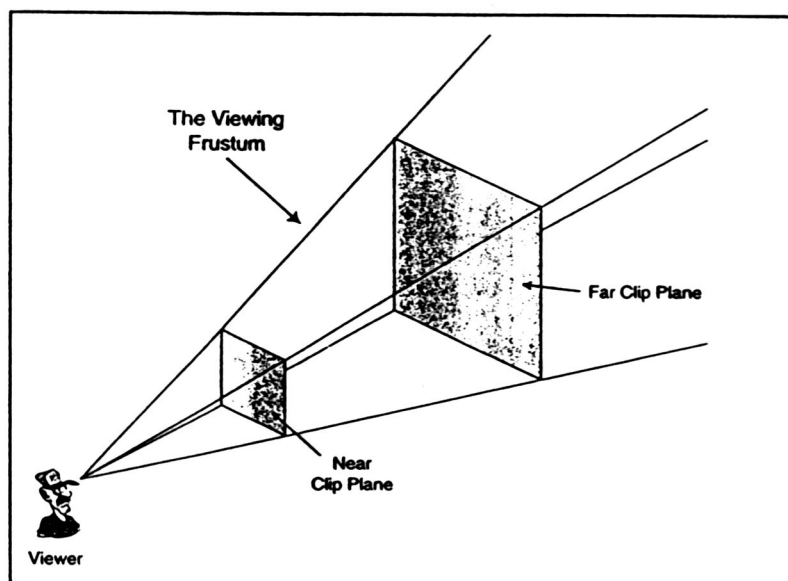
การทำงานในส่วนนี้ ก็ใช้ความสามารถของ Direct Graphic เป็นหลัก ซึ่งเราได้รวมไว้ในไลบรารีเกมเอนจินแล้ว แต่เนื่องจากเกมแบบ MMORPG นั้น มีลักษณะที่เด่นเป็นพิเศษก็คือ ผู้เล่นจะแทนตัวละครตัวหนึ่งในโลกจำลอง และจะมีผู้เล่นคนอื่นๆ มากมายเข้ามาอยู่ร่วมในโลกจำลองนี้ ทำให้เราจำเป็นต้องนำอัลกอริทึมต่างๆ มาใช้ในส่วนไคลเอนต์ของเรา โดยจะอธิบายต่อไปตามหัวข้อต่อไปนี้

- Viewing Frustum and Node Tree Engine
- Billboarding, Alpha Blending and Alpha Testing
- Third Person View

13.1 Viewing Frustum and Node Tree Engine

เนื่องจากโลกจำลองหรือแผนที่ในส่วนไคลเอนต์ของเกมของเรามีขนาดค่อนข้างใหญ่และมีรายละเอียดมาก ทำให้จำนวนโพลีกอนมากตาม ซึ่งการเรนเดอร์ทั้งแผนที่ทุกๆ ลูปเกมนั้นเป็นไปได้เพราะภาพที่ได้จะกระตุกมาก เนื่องจากหน่วยประมวลผลทางกราฟิกในการ์ดจอ (GPU : Graphics Processing Unit) ไม่สามารถทำงานได้ทัน ดังนั้นเราจึงต้องนำเทคนิค Viewing Frustum และ Node Tree มาใช้เพื่อให้การแสดงผลโลกที่มีจำนวนโพลีกอนมากๆ ของเราได้

Viewing Frustum คือ ปริมาตรสามมิติที่ประกอบด้วยระนาบ 6 ระนาบ โดยปริมาตรดังกล่าวจะมีความสัมพันธ์กับหน้ากล้อง หรือมุมมองผู้เล่น ดังรูปที่ 13-1



รูปที่ 13-1 Viewing Frustum

โดยระนาบทั้งหกด้านนี้เรียกว่า Clipping Planes ซึ่งเฟลนหรือระนาบดังกล่าว จะทำให้เราพิจารณาได้ว่าอ็อบเจ็กต์ใดบ้างที่ผู้ใช้เห็นในขณะนั้นและจำเป็นจะต้องทำการเรนเดอร์ ซึ่งเราสามารถคำนวณหาระนาบทั้งหกได้จากสูตร

```
D3DXMATRIX Matrix, matView, matProj;
```

```
m_Graphics->GetDeviceCOM()->GetTransform(D3DTS_PROJECTION, &matProj);
m_Graphics->GetDeviceCOM()->GetTransform(D3DTS_VIEW, &matView);
D3DXMatrixMultiply(&Matrix, &matView, &matProj);
```

```
// Calculate the planes
m_Planes[0].a = Matrix._14 + Matrix._13; // Near
m_Planes[0].b = Matrix._24 + Matrix._23;
m_Planes[0].c = Matrix._34 + Matrix._33;
m_Planes[0].d = Matrix._44 + Matrix._43;
D3DXPlaneNormalize(&m_Planes[0], &m_Planes[0]);
```

```
m_Planes[1].a = Matrix._14 - Matrix._13; // Far
m_Planes[1].b = Matrix._24 - Matrix._23;
m_Planes[1].c = Matrix._34 - Matrix._33;
m_Planes[1].d = Matrix._44 - Matrix._43;
D3DXPlaneNormalize(&m_Planes[1], &m_Planes[1]);
```

```
m_Planes[2].a = Matrix._14 + Matrix._11; // Left
m_Planes[2].b = Matrix._24 + Matrix._21;
m_Planes[2].c = Matrix._34 + Matrix._31;
m_Planes[2].d = Matrix._44 + Matrix._41;
D3DXPlaneNormalize(&m_Planes[2], &m_Planes[2]);
```

```
m_Planes[3].a = Matrix._14 - Matrix._11; // Right
m_Planes[3].b = Matrix._24 - Matrix._21;
m_Planes[3].c = Matrix._34 - Matrix._31;
m_Planes[3].d = Matrix._44 - Matrix._41;
D3DXPlaneNormalize(&m_Planes[3], &m_Planes[3]);
```

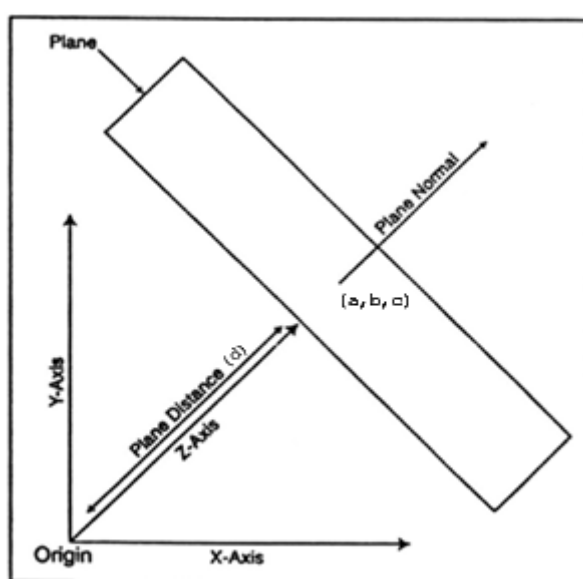
```

m_Planes[4].a = Matrix._14 - Matrix._12; // Top
m_Planes[4].b = Matrix._24 - Matrix._22;
m_Planes[4].c = Matrix._34 - Matrix._32;
m_Planes[4].d = Matrix._44 - Matrix._42;
D3DXPlaneNormalize(&m_Planes[4], &m_Planes[4]);

m_Planes[5].a = Matrix._14 + Matrix._12; // Bottom
m_Planes[5].b = Matrix._24 + Matrix._22;
m_Planes[5].c = Matrix._34 + Matrix._32;
m_Planes[5].d = Matrix._44 + Matrix._42;
D3DXPlaneNormalize(&m_Planes[5], &m_Planes[5]);

```

ค่า a , b , c และ d นั้นเป็นตัวแปรที่แทนระนาบ โดยค่า a , b , c คือจุด x , y , z ที่รังสีจากจุดกำเนิดตัดตั้งฉากกับระนาบ และ d คือระยะทางจากจุดกำเนิดไปยังจุดตัดนั้นบนระนาบ



รูปที่ 13-2 ตัวแปร a , b , c และ d ของระนาบ

ซึ่งการพิจารณาว่าจุดหนึ่งๆ หรือเวกเตอร์หนึ่งๆ นั้นอยู่ในฟรอสตัมหรือไม่นั้น ทำได้การหา Dot Product ของจุดนั้นกับแต่ละระนาบ โดยใช้คำสั่ง

```

FLOAT D3DXPlaneDotCoord(
    CONST D3DXPLANE * pP,    //D3DXPLANE to check
    CONST D3DVECTOR* pV);    //Point to check

```

โดยฟังก์ชันนี้จะคืนค่ามาเป็นระยะทางระหว่างจุดกับระนาบ ซึ่งถ้าได้ค่าเป็นบวกแสดงว่าจุดอยู่หน้าระนาบ ถ้าเป็นลบแสดงว่าจุดอยู่หลังระนาบ เมื่อเราทำการหา Dot Product ของจุดใดๆ กับระนาบทั้งหมดแล้ว เราก็จะรู้ได้ว่าจุดๆ นั้นอยู่นอกหรืออยู่ในฟรอสตัม โดยทำให้ฟังก์ชันตรวจสอบดังนี้

```

BOOL cFrustum::CheckPoint(float XPos, float YPos, float ZPos)
{

```

```

short i;

// Make sure point is in frustum
for(i=0;i<6;i++) {

    if(D3DXPlaneDotCoord(&m_Planes[i],&D3DXVECTOR3(XPos,YPos,ZPos))<0.0f)
        return FALSE;
}

return TRUE;
}

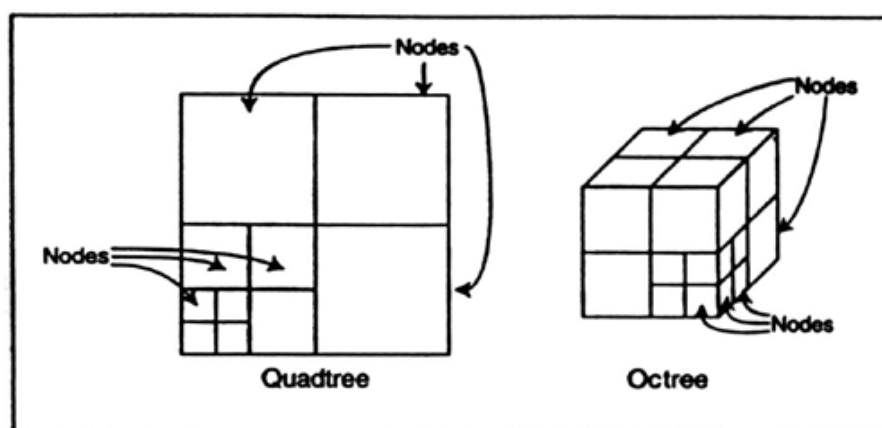
```

ถ้าจุดนี้อยู่ด้านหลังระนาบใดระนาบหนึ่งแสดงว่าจุดนั้นอยู่นอกพริสทัม หรือถ้าเราต้องการหาว่าโพลีกอนใดอยู่ในพริสทัมหรือไม่ เราก็ทำการตรวจสอบเช่นเดียวกันนี้กับจุดทั้ง 3 จุด ของโพลีกอนว่า ถ้ามีจุดใดจุดหนึ่งอยู่ในพริสทัมโพลีกอนนั้นถือว่าอยู่ในพริสทัมและต้องเรนเดอร์

จากแนวคิดเรื่อง Viewing Frustum นั้นทำให้้อบเจ็กต์หรือโพลีกอนที่อยู่ในปริมาตรนี้เท่านั้นที่จะถูกเรนเดอร์ ช่วยให้เราสามารถลดการปริมาณการประมวลผลในขณะเรนเดอร์ไปได้มากที่สุดทีเดียว แต่อย่างไรก็ดี เราหลีกเลี่ยงไม่ได้ที่จะต้องสแกนผ่านทุกๆ โพลีกอนในโมเดลของเราแล้วพิจารณาว่าโพลีกอนนั้นอยู่ในพริสทัมหรือไม่ และจะต้องสแกนใหม่ทุกๆ ลูปการเรนเดอร์ ซึ่งนั่นก็เป็นการเสียเวลาอย่างมากทีเดียว

ดังนั้นแทนที่เราจะสแกนผ่านเพื่อพิจารณาทุกๆ โพลีกอน เราจะนำเทคนิค Node Tree เข้ามาช่วย โดยเราจะทำการแบ่งโมเดลขนาดใหญ่ของเราออกเป็นส่วนย่อยๆ เรียกว่า โหนด และจัดเรียงโหนดเหล่านั้นให้อยู่ในรูปของ Tree จากนั้นเราจะสามารถสแกนผ่าน Tree ได้โดยง่าย และทำการเรนเดอร์โหนดที่มีบางส่วนอยู่ในพริสทัมเลย ซึ่งช่วยเราลดเวลาในการสแกนไปได้มากที่สุดทีเดียว

โดยลักษณะการแบ่งโมเดลออกเป็นโหนดย่อยๆนั้น สามารถทำได้ 2 แบบด้วยกันคือ แบบ Quadtree และแบบ Octree ดังรูป



รูปที่ 13-3 การแบ่งโหนดแบบ Quadtree และ Octree

จะเห็นว่าแบบ Quadtree จะทำการแบ่งโหนดใหญ่ออกเป็นครั้งละ 4 โหนดย่อย โดยจะแบ่งเฉพาะระนาบ XZ (2D-space) ดังนั้นแบบนี้จะไม่เหมาะกับเกมที่มีการเปลี่ยนตำแหน่งในแนวแกน Y

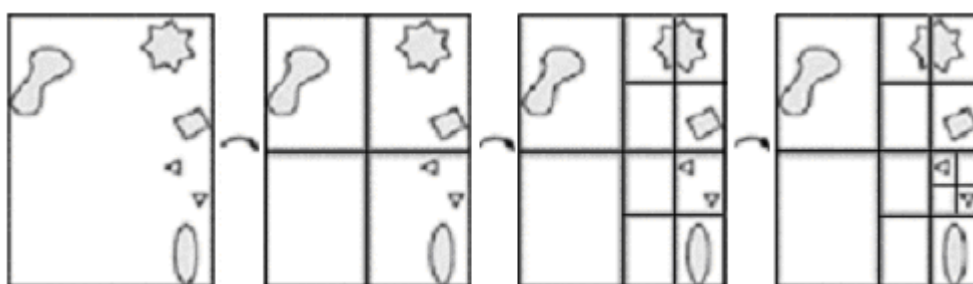
มากๆ และอีกแบบคือแบบ Octree จะแบ่งโหนดใหญ่ออกเป็นโหนดรูปทรงสี่เหลี่ยมครั้งละ 8 โหนดย่อย ในระนาบ XYZ (3D-space) ซึ่งแต่ละโหนดจะเป็นตัวแทนของกลุ่มโพลีกอนที่อยู่บริเวณที่กำหนด

โดยโหนดใหญ่จะมีความสัมพันธ์กับโหนดย่อยที่ถูกแบ่งออกนั้น ในลักษณะพ่อกับลูก (Parent-Child) ซึ่งโหนดใหญ่สุดหรือโมเดลทั้งโมเดล เรียกว่า Root Node

การสร้าง Node Tree นั้นจะทำเพียงครั้งเดียวเท่านั้น ฉะนั้นความเร็วซ้ำในการสร้างจึงไม่ใช่ปัญหาสำคัญ โดยวิธีการสร้างโหนดคือ

อันดับแรกเราจะกำหนด Bounding Box หรือกล่องบริเวณที่ต้องการจัดกลุ่มโพลีกอน จากนั้นจะสแกนดูว่ามีโพลีกอนใดอยู่หรือมีบางส่วนอยู่ในกล่องนี้บ้าง (หนึ่งโพลีกอนอาจอยู่ได้หลายโหนด)

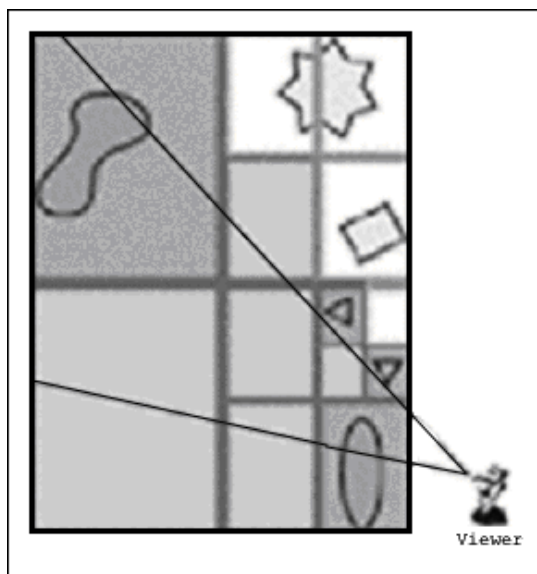
หลังจากนั้นทำการแบ่งกล่องออกเป็นกล่องย่อยๆ ทำการสแกนเช่นเดียวกันอีกครั้ง ทำการแบ่งโหนดที่ยังมีขนาดใหญ่หรือมีโพลีกอนมากกว่ากำหนดไปเรื่อยๆ จนกว่าจะได้ขนาดโหนดที่เล็ก และมีจำนวนโพลีกอนไม่มากเกินไปตามที่ต้องการ ส่วนโหนดใดที่ไม่มีโพลีกอนอยู่เลยเราจะทำการละทิ้งโหนดนั้นไป



รูปที่ 13-4 การแบ่งโหนดออกเป็นโหนดย่อยๆ

เมื่อเสร็จแล้วเราก็จะได้ Node Tree ที่มีขนาดโหนดตามที่ต้องการ จากนั้นเมื่อเราจะเรนเดอร์ เราก็จะทำการสแกนโหนด โดยจะเริ่มที่ Root Node ถ้าโหนดนั้นมีส่วนหนึ่งส่วนใดอยู่ในฟรอสตัม จะทำการสแกนลงไปโหนดลูก ถ้าไม่มีส่วนใดอยู่ในฟรอสตัมก็จะไม่สนใจโหนดนั้น

จากนั้นจะทำการสแกนต่อไปยังโหนดลูก จนกระทั่งไม่มีลูกต่อแล้ว จะสิ้นสุดการสแกน และทำการคัดลอกข้อมูลโพลีกอนจากทุกๆ โหนดสุดท้ายที่มีส่วนอยู่ในฟรอสตัมมารวมกันไว้ในบัฟเฟอร์ จากนั้นจะทำการเรนเดอร์บัฟเฟอร์นั้น



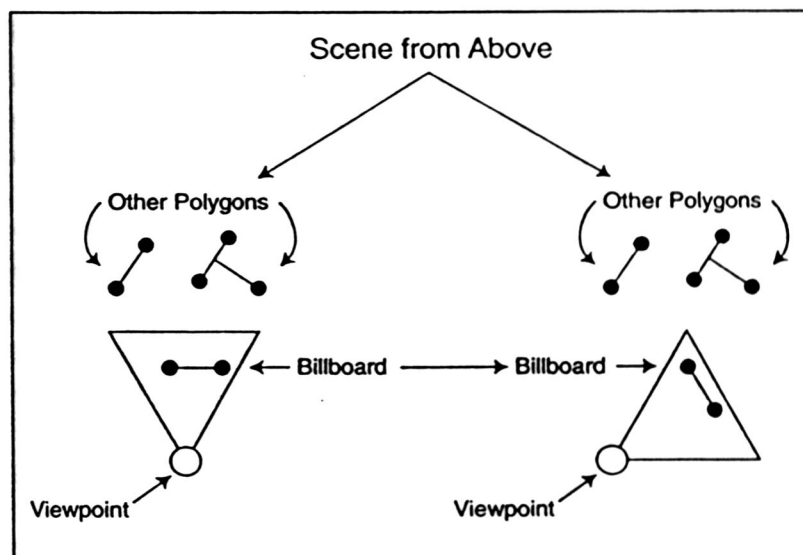
รูปที่ 13-5 โหนดที่มีบางส่วนอยู่ในฟรอสตัมจะถูกเรนเดอร์

จากการใช้เทคนิค Viewing Frustum และ Node Tree จะทำให้เราสามารถเรนเดอร์โมเดลแผนที่ที่มีขนาดใหญ่และมีโพลีกอนมากได้ โดยช่วยแก้ปัญหาโมเดลขนาดใหญ่ไปได้มากทีเดียว

13.2 Billboard ,Alpha blending ang Alpha Testing

จากการที่เกม MMORPG นี้จะต้องมีผู้เล่นจำนวนมากอยู่ในโลกใบเดียวกัน และในหนึ่งหน้าจอที่ผู้เล่นเห็นอาจมีผู้เล่นอยู่พร้อมกันจำนวนมาก ดังนั้นการที่จะได้ภาพตัวละครสามมิติที่สวยงาม จำเป็นต้องใช้โพลีกอนต่อหนึ่งตัวละครมากๆ อาจเป็นสิ่งที่เกินกำลังต่อการเรนเดอร์ ดังนั้นเราจะใช้ภาพตัวละครสองมิติเข้ามาอยู่ในโลกสามมิติ โดยสร้างโพลีกอน 2 อันประกอบกันเป็นรูปสี่เหลี่ยมเพื่อปะภาพตัวละคร และใช้เทคนิค Billboarding เข้ามาช่วย โดยการทำให้รูปสี่เหลี่ยมของเราหันหน้าหากล้องตลอดเวลา ไม่ว่ากล้องจะไปทางไหนก็ตาม ก็จะเห็นภาพบนรูปสี่เหลี่ยมหรือตัวละครของเราได้อย่างถูกต้อง ไม่บิดเบี้ยว

โดยเราจะต้องหา World Transform Matrix มาหมุนสี่เหลี่ยมให้หันหน้าเข้ากล้อง ซึ่งคำนวณได้จาก View Transform Matrix ว่าตอนนี้กล้องหมุนทำมุมอย่างไร เพื่อคำนวณให้สี่เหลี่ยมของเราหมุนกลับก็จะได้ภาพที่หันหน้าหากล้องตลอดเวลา



รูปที่ 13-6 การทำ Billboarding

```
// g_D3DDevice = pre-initialized device object

D3DXMATRIX matBillboard;

D3DXMatrixIdentity(&matBillboard);

// calculate billboarding matrix
g_D3DDevice->GetTransform(D3DTS_VIEW, &matView);
D3DXMatrixTranspose(&matTransposed, &matView);
matTransposed._41 = matTransposed._42 = matTransposed._43 =
matTransposed._14 = matTransposed._24 = matTransposed._34 = 0.0f;

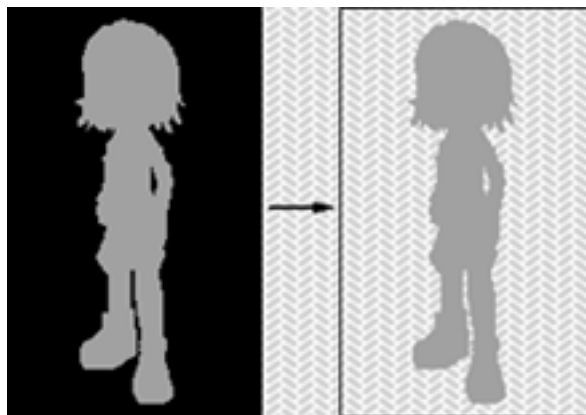
// Apply billboard matrix to world transform matrix
g_D3DDevice->SetTransform(D3DTS_WORLD, &matTempWorld);
```

หรือถ้าใช้เกมเอนจินของเราจะใช้คำสั่งว่า

```
// m_WorldPos = pointer to pre-initialized cWorldPosition object
// m_Graphics = pointer to pre-initialized cGraphics object

m_WorldPos->EnableBillboard(TRUE);
m_Graphics-> SetWorldPosition(&m_WorldPos);
```

และจากโพลีกอนที่ทำเป็นรูปสี่เหลี่ยมนั้น เราจะทำการแปะภาพตัวละครลงไป ซึ่งเราจะต้องทำให้ส่วนที่ไม่ใช่ตัวละครโปร่งใส เพื่อให้เห็นแต่ตัวละครจริงๆ โดยเราจะนำเทคนิค Alpha Testing เข้ามาช่วย คือทำการกำหนด Color Key ที่จะให้ทำโปร่งใส ดังในรูปเราจะให้สีดำโปร่งใสเป็นต้น



รูปที่ 13-7 การทำ Alpha Testing

วิธีการทำ Alpha Testing คือเราจะใช้คำสั่ง D3DXCreateTextureFromFileEx โดยมีลักษณะคำสั่งดังรูป

```
HRESULT D3DXCreateTextureFromFileEx(
    LPDIRECT3DDEVICE8 pDevice,
    LPCTSTR pSrcFile,
    UINT Width,
    UINT Height,
    UINT MipLevels,
    DWORD Usage,
    D3DFORMAT Format,
    D3DPOOL Pool,
    DWORD Filter,
    DWORD MipFilter,
    D3DCOLOR ColorKey,
    D3DXIMAGE_INFO* pSrcInfo,
    PALETTEENTRY* pPalette,
    LPDIRECT3DTEXTURE8* ppTexture
);
```

ซึ่งออกจะวุ่นวาย ดังนั้นเราจะใช้เป็นคำสั่งของเกมเอนจินของเราแทน โดยใช้คำสั่งคือ

```
m_Texture->Load(m_Graphics, "texture.bmp", 0xFFFFFFFF);
```

เป็นการสั่งโหลดภาพจากไฟล์ texture.bmp โดยกำหนด Color Key เป็นสีดำ และก่อนการเรนเดอร์เราจะต้องทำการ Enable Alpha Testing เสียก่อนโดยใช้คำสั่ง

```
m_Graphics->EnableAlphaTesting(TRUE);
```

แต่ข้อเสียของการทำ Alpha Testing คือเราสามารถกำหนดสีที่จะให้โปร่งใสได้สีเดียวเท่านั้น เช่น ถ้าเรากำหนดให้สีดำโปร่งใส ถ้าสีเกือบๆ ดำ ไม่ดำสนิทก็จะเห็นขึ้นเป็นขอบสีเกือบดำนั้น ซึ่งเราจะต้องใช้ Alpha Blending เข้ามาช่วย เพื่อเกลี่ยสีที่เกือบดำนั้นให้โปร่งแสงด้วย โดยวิธีใช้อย่างง่ายๆ นั้น เราไม่ต้องทำอะไรมากมาย เนื่องจากเกมเอนจินของเราโดยค่าเริ่มต้นแล้วได้กำหนดให้ทำการ Blend สีไปยังค่าสีของ Vertex อยู่แล้ว เราไม่ต้องใช้คำสั่งในการ Blend ด้วยตนเองเลย เพียงแต่กำหนดสีให้กับ Vertex ตามสีที่ต้องการเท่านั้น จากนั้นก่อนการเรนเดอร์เราก็ทำการสั่ง Enable Alpha Blending ด้วย

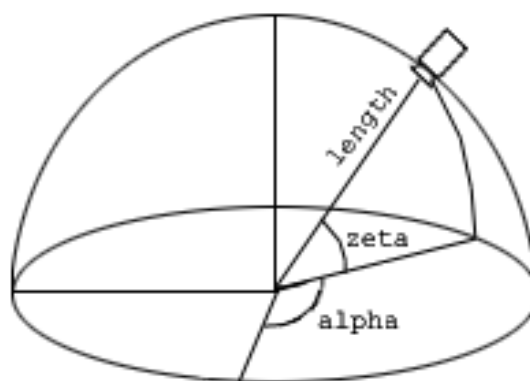
```
m_Graphics->EnableAlphaBlending(TRUE);
```

จากการใช้ Billboarding, Alpha Blending และ Alpha Testing นี้ ทำให้เราสามารถแสดงผลภาพตัวละครสองมิติ ในโลกสามมิติของเราได้แล้ว

13.3 Third- Person View

มุมมองของกล้องที่ใช้ในเกมของเรานั้นเป็นแบบมุมมองบุคคลที่สาม คือผู้เล่นเกมจะมองเห็นตัวละครของตนที่กำลังกระทำอะไรกับใครอยู่บ้าง โดยเราสามารถปรับมุมมองกล้องได้ 3 ลักษณะ คือ การหมุนกล้องวนรอบแกน Y การก้มเงยกล้อง และการซูมเข้าออก โดยเราจะทำการสร้างคลาสกล้องขึ้นมาใหม่เก็บตัวแปรเบื้องต้นสามตัว คือ Length, Zeta และ Alpha โดยที่

- Length จะใช้ในกรณีซูมเข้าออก
- Zeta ใช้ในกรณีก้มเงยกล้อง
- Alpha ใช้ในกรณีหมุนกล้อง



รูปที่ 13-8 ตัวแปรมุมมองกล้อง Length, Zeta และ Alpha

แต่เนื่องจากภาพตัวละครของเราเป็นสองมิติ การจะหมุนมุมมองแล้วเจอบทภาพทิศทางเดิมตลอดนั้นไม่ควรเกิดขึ้น ดังนั้นเราจะต้องทำภาพตัวละครในทิศทางต่างๆ ซึ่งในที่นี้ตัวละครของเรามีแปดทิศทาง ซึ่งเราจะต้องเตรียมภาพในแปดทิศทางนี้ สำหรับทุกๆ ทิศทาง ไม่ว่าจะเป็น เดิน ต่อสู้ บาดเจ็บ เป็นต้น จะเห็นได้ว่าเราจะต้องใช้ภาพจำนวนมหาศาลเลยทีเดียว

ซึ่งถ้าเราจะใช้วิธี สร้าง Texture Buffer จำนวนมากๆ และใช้การสลับ Texture ตลอดเวลาที่หมุนกล้องนั้นเป็นไปได้ เพราะการสลับ Texture หนึ่งครั้งนั้นใช้เวลาและทรัพยากรมาก เพื่อแก้ปัญหานี้ เราจะใช้เทคนิคที่คล้ายคลึงกับที่ใช้ในเกมสองมิติเข้ามาช่วย นั่นก็คือวิธี Sprite Animation โดยจะเป็นการรวมเอาภาพทุกภาพ ทุกทิศทางมาต่อรวมกันเป็นภาพเดียว ทำให้เราไม่ต้องสลับ Texture และเราจะใช้การเปลี่ยนค่าใน UV-Coordinate หรือ Texture Coordinate แทนนั่นเอง

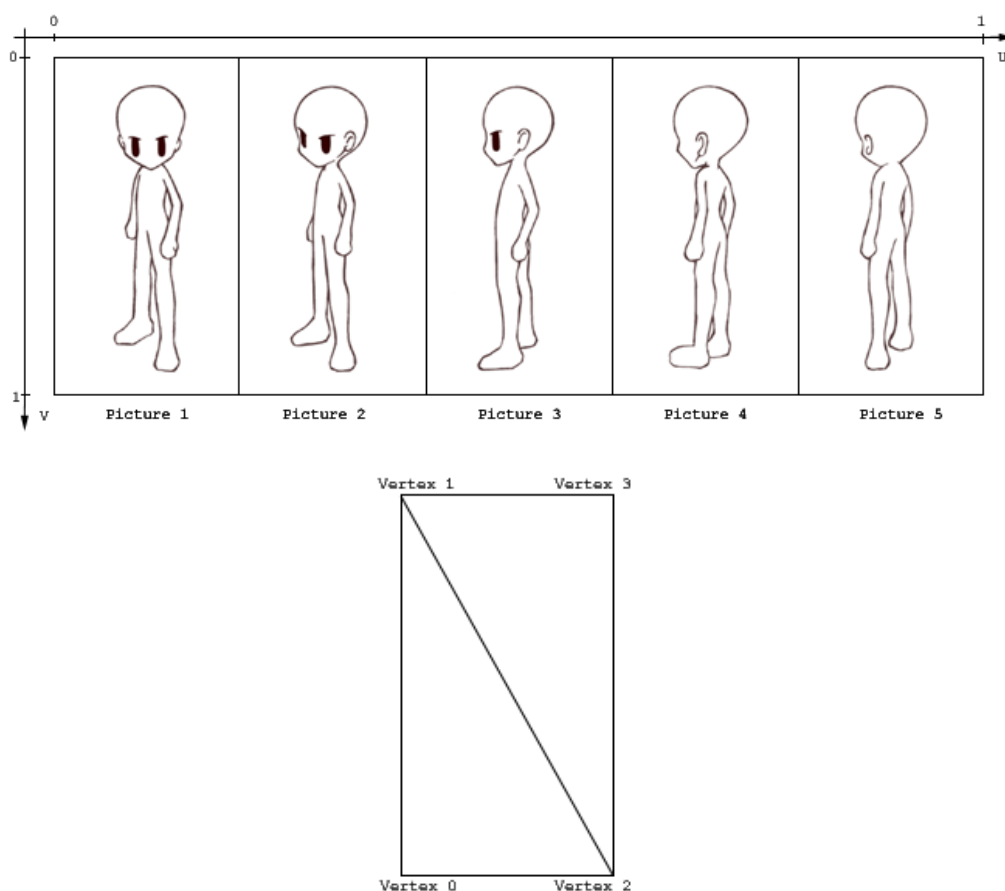
เช่นถ้าเรามีภาพดังรูปที่ 13-9 ซึ่งเป็นภาพทำขึ้น 5 ทิศทาง และเราจะทำการ Mirror ภาพที่ 2, 3 และ 4 เพื่อให้ได้ภาพทั้งหมดแปดทิศ

ถ้าต้องการแสดงภาพด้านหน้า เราต้องทำการกำหนดค่า UV ของ Vertex ทั้ง 4 จุดดังนี้

```
vertex0.u = 0.0f;          vertex0.v = 1.0f;
vertex1.u = 0.0f;          vertex1.v = 0.0f;
vertex2.u = 1.0f / 5.0f;   vertex2.v = 1.0f;
vertex3.u = 1.0f / 5.0f;   vertex3.v = 0.0f;
```

และถ้าต้องการแสดงภาพด้านข้างที่หันหน้าไปทางซ้าย ค่า UV จะเป็นดังนี้

```
vertex0.u = 2.0f / 5.0f;   vertex0.v = 1.0f;
vertex1.u = 2.0f / 5.0f;   vertex1.v = 0.0f;
vertex2.u = 3.0f / 5.0f;   vertex2.v = 1.0f;
vertex3.u = 3.0f / 5.0f;   vertex3.v = 0.0f;
```



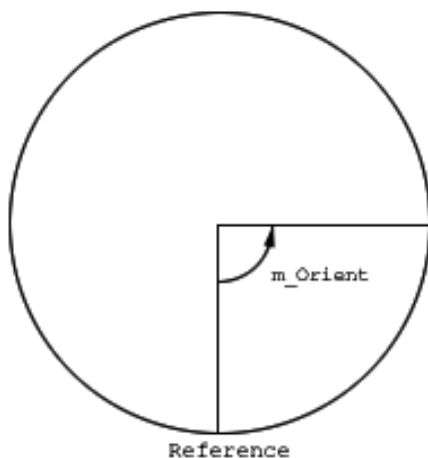
รูปที่ 13-9 บน ภาพยืนในทิศต่างๆ

รูป 13-10 ล่าง Vertex ที่ใช้ทำรูปสี่เหลี่ยมเพื่อแสดงภาพ

ซึ่งในการใช้งานจริงจะมีภาพมากกว่านี้มากนัก เนื่องจากเราต้องใช้หลายๆภาพ ทำเป็นภาพเคลื่อนไหวในหนึ่งท่าทาง แต่การกำหนดค่า UV ก็จะหลักการเดียวกันกับที่ใช้กำหนดในข้างต้นนี้

ในขั้นตอนต่อไป การที่เราจะรู้ได้ว่าในขณะนั้นจะต้องแสดงภาพทิศทางไหน เราจะต้องรู้ก่อนว่าตัวละครหันด้านใดให้กับกล้อง เพื่อที่เราจะรู้ได้ว่าตัวละครหันด้านใดให้กับกล้องนั้น เราจะต้องใช้ตัวแปรอีก 2 ตัว เข้ามาช่วย ตัวแปรแรกคือ ตัวแปรที่ใช้เก็บทิศทางของคน ว่าคนนี้หันหน้าไปทิศทางไหน

เช่น ถ้าคนหันหน้าไปทางขวา ดังรูป เราจะเก็บค่ามุมเทียบกับทิศอ้างอิง จะได้ค่า m_Orient ของตัวละคร ตัวนี้เป็น 90 องศา



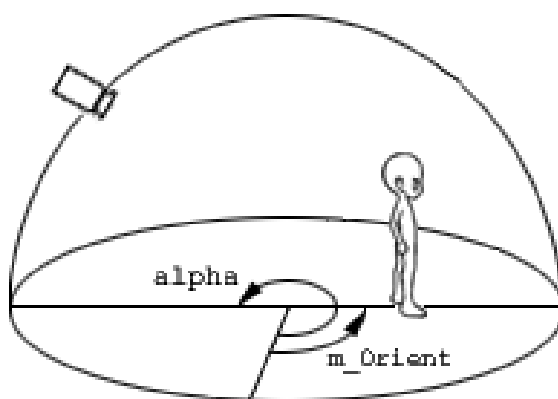
รูปที่ 13-11 การเก็บค่าทิศทางของตัวละคร

เมื่อเรารู้ว่าตัวละครของเราหันไปทางทิศใดแล้ว แต่เรายังไม่สามารถแสดงภาพที่ถูกต้องได้ เนื่องจากยังไม่ใช้ทิศทางที่หันหากล้อง ดังนั้นเราจะนำค่าทิศทางของกล้องหรือค่า Alpha มาใช้เป็นตัวแปรที่สอง โดยเราจะรู้ได้ว่าคนหันหน้าทิศใดให้กับกล้อง เราจะนำค่า Alpha ของกล้อง มาหาความแตกต่างกับทิศทาง m_Orient ของตัวละคร เช่นตัวอย่างในรูป

$m_Orient = 90$ เนื่องจากคนหันไปทางขวา ทำมุม 90 องศา กับทิศอ้างอิง

$Alpha = 270$ กล้องหมุนไป 270 องศา

จะได้ว่าภาพที่เราต้องแสดงคือภาพที่มีทิศทาง 180 องศาหรือ เห็นด้านหลังของตัวละครนั่นเอง



รูป 13-12 การคำนวณหาภาพตัวละครในทิศทางที่ควรเห็น